

Name: Kassim Musa Abass  
Reg: SPH32/3338/2025  
Program: Embedded Systems Intro

## Week 4: Pointers & Memory Management

### Objective:

To demonstrate a strong understanding of C pointers, dynamic memory allocation, and memory-mapped I/O required for embedded programming.

### Introduction:

In embedded systems programming, direct memory manipulation is essential for reading from and writing to hardware registers. This report illustrates the fundamentals of pointers and simulates accessing memory-mapped hardware.

### 1. Theoretical Concepts

**Pointers and Arrays:** Arrays in C are closely related to pointers; an array name is essentially a pointer to its first element. We demonstrated pointer arithmetic to traverse arrays.

**Dynamic Memory:** Using `malloc()` allows for runtime memory allocation, which must be carefully managed and freed using `free()` to avoid memory leaks.

**Memory-Mapped I/O:** Microcontrollers map their peripheral hardware (like GPIO pins) to specific memory addresses. By casting these addresses to volatile pointers, we can control hardware by simply reading or writing to these memory locations.

### 2. Source Code

```
mylandingpage > reports > week4 > week4_pointers.c > ...
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdint.h>
4
5 // A fake memory buffer to simulate hardware registers since we are running on a PC
6 uint32_t fake_memory[1024];
7
8 int main() {
9     printf("=== Week 4: Pointers & Memory Management ===\n\n");
10
11     // 1. Pointer Basics
12     int value = 42;
13     int *ptr = &value;
14     printf("1. Pointer Basics:\n");
15     printf("   Value: %d\n", value);
16     printf("   Address of Value: %p\n", (void*)&value);
17     printf("   Pointer holds address: %p\n", (void*)ptr);
18     printf("   Dereferenced pointer value: %d\n\n", *ptr);
19
20     // 2. Arrays and Pointers
21     printf("2. Arrays and Pointer Arithmetic:\n");
22     int arr[3] = {10, 20, 30};
23     int *arr_ptr = arr; // points to first element
24     for(int i = 0; i < 3; i++) {
25         printf("   arr[%d] = %d at address %p\n", i, *(arr_ptr + i), (void*)(arr_ptr + i));
26     }
27     printf("\n");
28
29     // 3. Dynamic Memory Allocation
30     printf("3. Dynamic Memory Allocation:\n");
31     int *dyn_arr = (int*)malloc(3 * sizeof(int));
32     if (dyn_arr == NULL) {
33         printf("   Memory allocation failed!\n");
34     }
35 }
```

### 3. Output Results

```
(base) qassim@mr-ceo:~/mylandingpage/reports/week4$ cd /home/qassim/mylandingpage/reports/week4
gcc week4_pointers.c -o week4_pointers && ./week4_pointers
== Week 4: Pointers & Memory Management ==

1. Pointer Basics:
   Value: 42
   Address of Value: 0x7ffeb9a239d4
   Pointer holds address: 0x7ffeb9a239d4
   Dereferenced pointer value: 42

2. Arrays and Pointer Arithmetic:
   arr[0] = 10 at address 0x7ffeb9a239c8
   arr[1] = 20 at address 0x7ffeb9a239cc
   arr[2] = 30 at address 0x7ffeb9a239d0

3. Dynamic Memory Allocation:
   Allocated dynamic memory at 0x558719e456b0
   Values: 100, 200, 300
   Memory freed.

4. Memory-Mapped I/O Simulation:
   Wrote 0x01 to GPIO simulated register at 0x5586dbac7060
(base) qassim@mr-ceo:~/mylandingpage/reports/week4$
```

### 4. Reflection

A key challenge is ensuring that pointers always point to valid memory addresses. Dereferencing a null or uninitialized pointer can cause a segmentation fault on a PC or completely crash a microcontroller. Using the 'volatile' keyword is critical for memory-mapped I/O because it prevents the compiler from optimizing away reads/writes to registers that might be changed by hardware.